Assignment 8

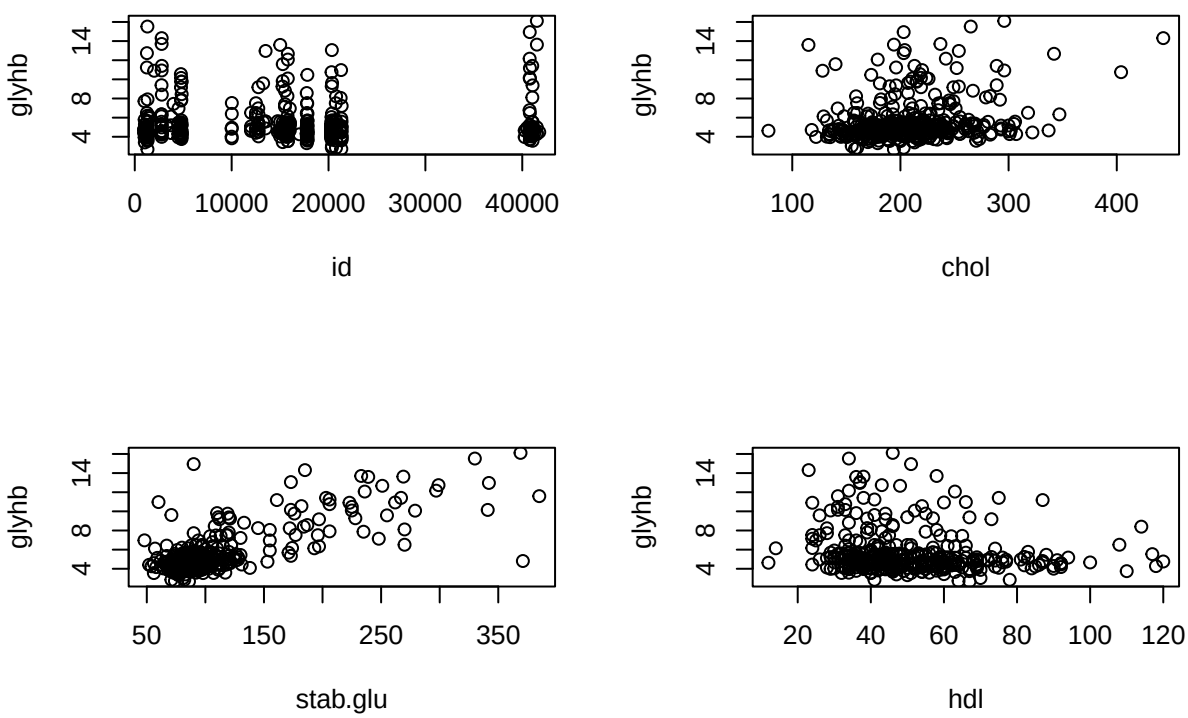
Chul Hee Kim

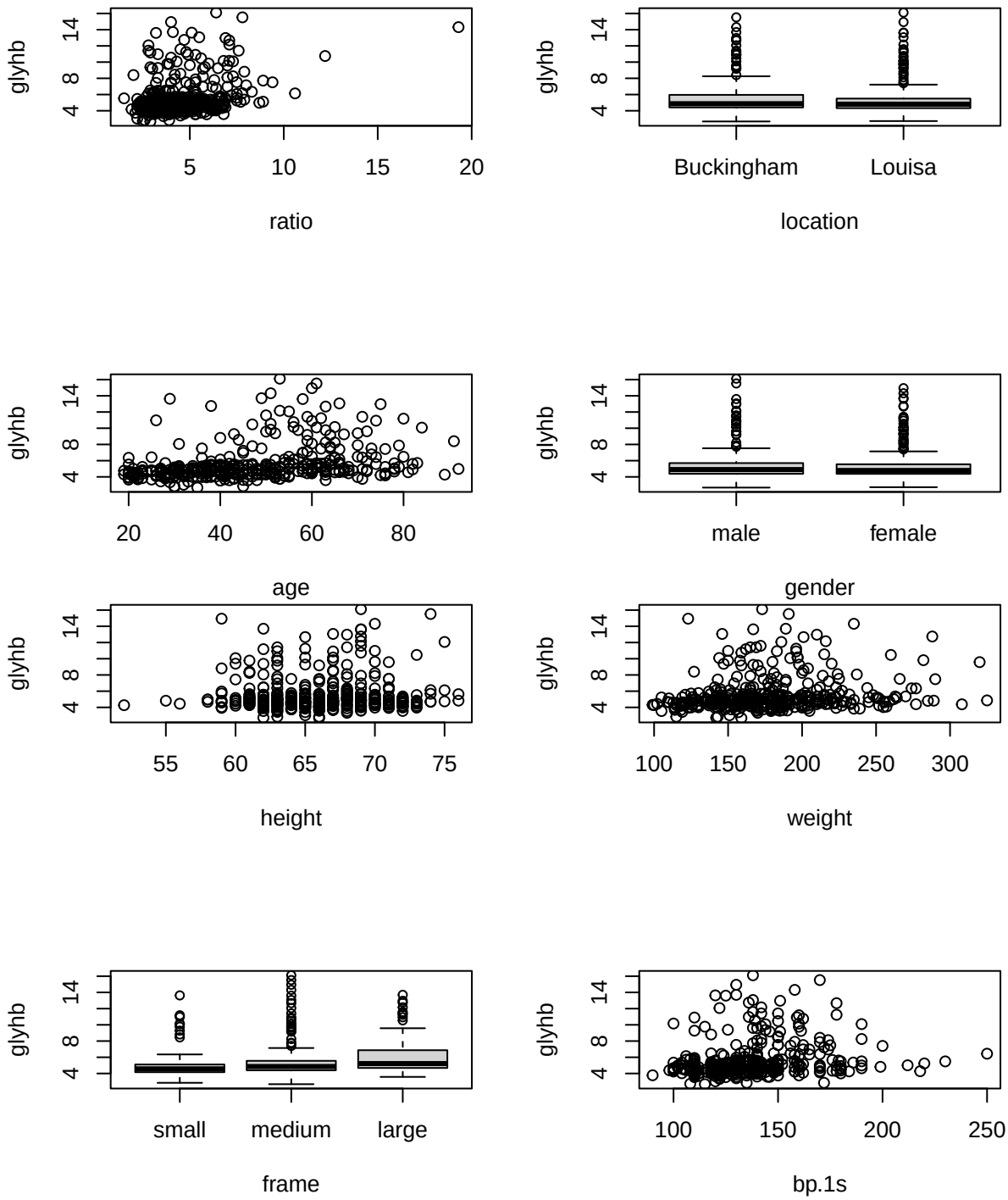
2021 11 5

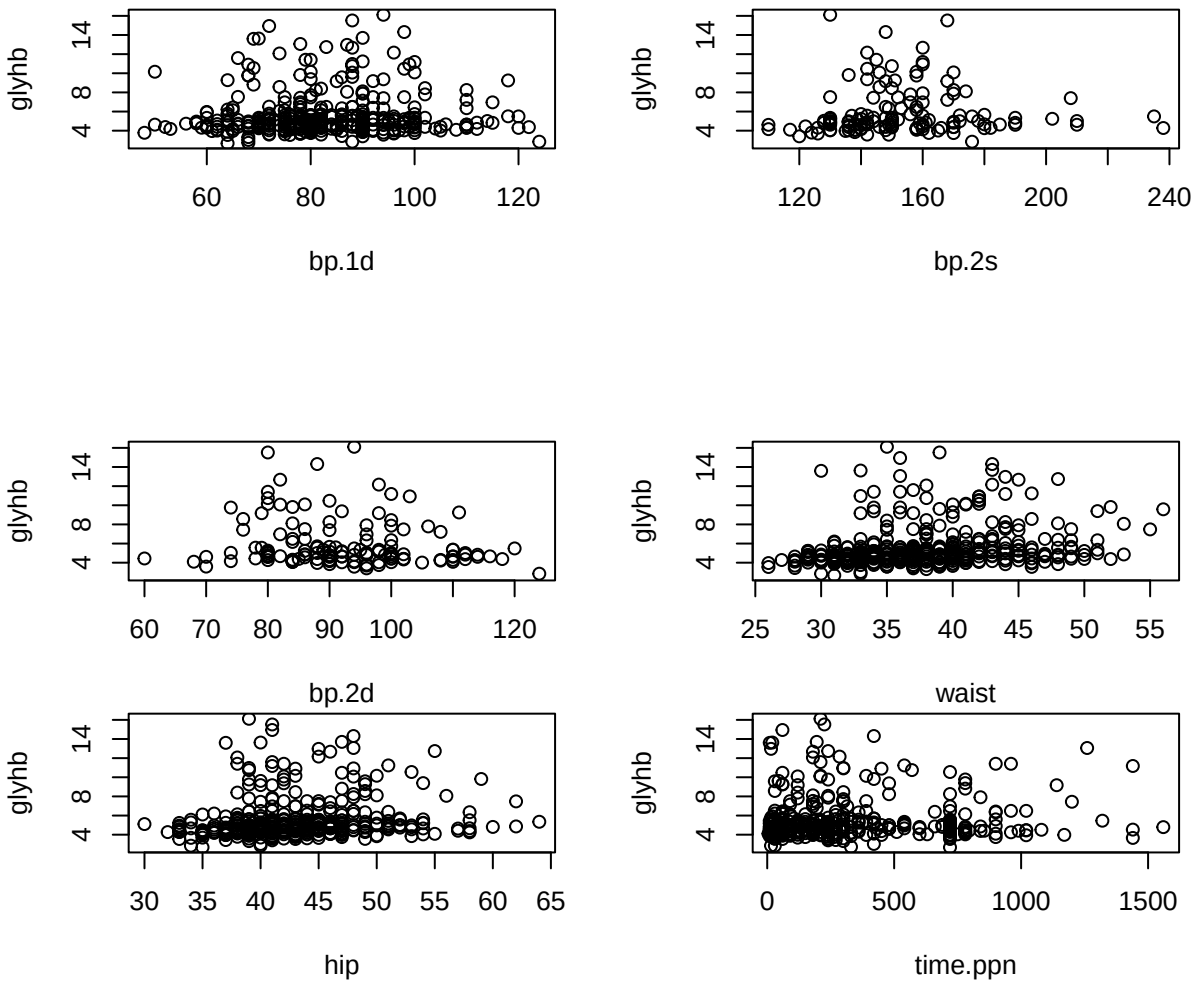
Problem 1

a-)

```
data(diabetes)
x <- diabetes[,-6]
par(mfrow=c(2,2))
for (i in 1:18){
  plot(x[,i], diabetes$glyhb, xlab=colnames(x)[i], ylab="glyhb")
}
```



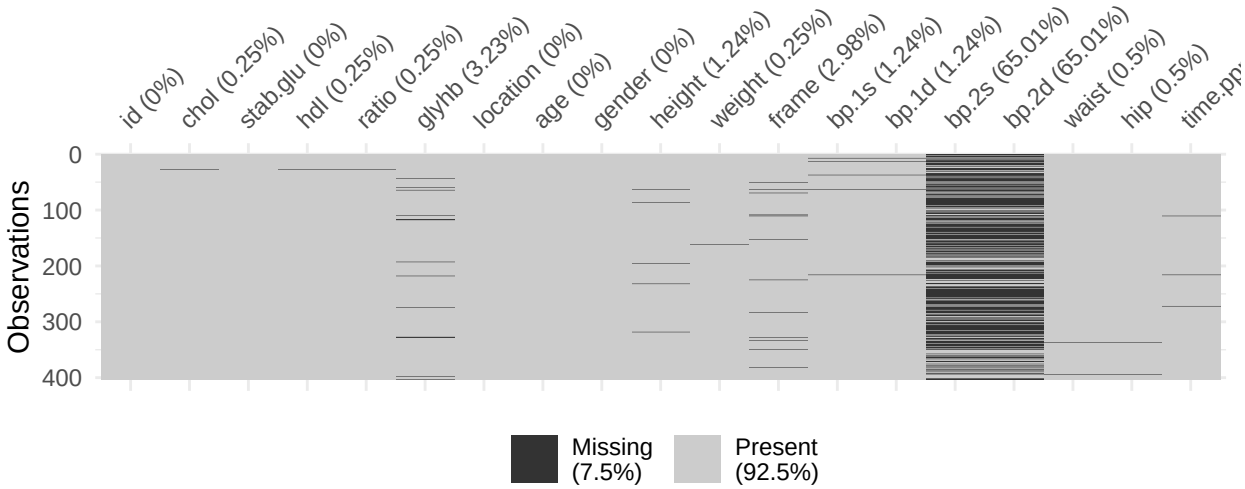

glyhb
ratio
location
Buckingham
Louisa
age
gender
male
female
height
weight
frame
small
medium
large
bp.1s



Among the 18 predictors, it seems that only 'stab.glu' has the apparent positive relationship with the response 'glyhb'. We also observe that 'age' or 'frame' might have significant relationship, but it is not very clear in the plots.

b-)

```
vis_miss(diabetes)
```



We see that there are a bunch of NAs for 'bp.2s' and 'bp.2d', so we remove these two predictors since removing them by rows will result in 262 fewer observations, which is 65% of the total observations. Next, we find that there are some remaining NAs, which can be eliminated by *na.omit* function.

```
diabetes2 <- diabetes[,-c(15,16)]
diabetes2 <- na.omit(diabetes2)
table(is.na(diabetes2))
```

```
##
## FALSE
##  6222
```

We finally see that there is no more NA in the data.

c-)

```
f1 <- lm(glyhb ~ ., data=diabetes2)
summary(f1)
```

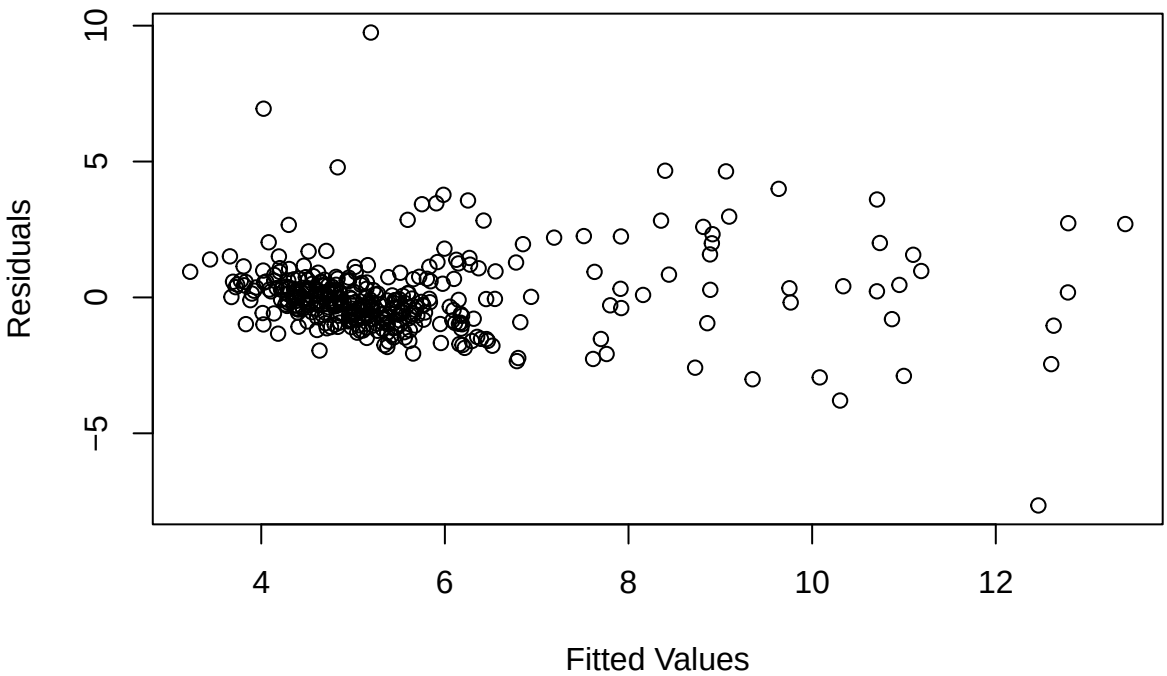
```
##
## Call:
## lm(formula = glyhb ~ ., data = diabetes2)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -7.6538 -0.6960 -0.1601  0.4650  9.7461
##
## Coefficients:
##                    Estimate   Std. Error t value Pr(>|t|)
## (Intercept)    -2.198326318  2.393219745  -0.919   0.3590
## id              0.000006213  0.000007168   0.867   0.3867
## chol            0.004888073  0.003363961   1.453   0.1471
## stab.glu        0.027524900  0.001528556  18.007   <2e-16 ***
## hdl            -0.003249383  0.010433486  -0.311   0.7557
## ratio           0.087846524  0.114331059   0.768   0.4428
## locationLouisa -0.287416020  0.172402351  -1.667   0.0964 .
## age             0.013502220  0.006065053   2.226   0.0266 *
## genderfemale    0.110178012  0.253468458   0.435   0.6641
## height          0.023340248  0.030644352   0.762   0.4468
## weight         -0.003916747  0.005265321  -0.744   0.4575
## framemedium     0.078421958  0.200995926   0.390   0.6967
## framelarge     -0.191563246  0.267674280  -0.716   0.4747
## bp.1s           0.002723695  0.004878065   0.558   0.5770
## bp.1d          -0.001576677  0.007578375  -0.208   0.8353
## waist           0.029552211  0.030814035   0.959   0.3382
## hip             0.013122690  0.035138376   0.373   0.7090
## time.ppn        0.000577626  0.000249904   2.311   0.0214 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.439 on 348 degrees of freedom
## Multiple R-squared:  0.6033, Adjusted R-squared:  0.584
## F-statistic: 31.14 on 17 and 348 DF,  p-value: < 2.2e-16
```

```
ols_step_both_p(f1, prem=0.05)
```

```
##
```

```
##                                Stepwise Selection Summary
## ---------------------------------------------------------------------------------------
##                        Added/                    Adj.
## Step    Variable     Removed     R-Square    R-Square     C(p)         AIC        RMSE
## ---------------------------------------------------------------------------------------
##    1    stab.glu    addition       0.549       0.548    33.7350    1339.6866    1.5005
##    2      chol      addition       0.572       0.569    15.6900    1322.6615    1.4640
##    3      age       addition       0.582       0.579     8.5560    1315.6535    1.4481
##    4     ratio      addition       0.588       0.584     5.1430    1312.2080    1.4393
##    5    time.ppn    addition       0.594       0.589     1.9410    1308.8983    1.4309
## ---------------------------------------------------------------------------------------
```

```
plot(f1$fitted.values, f1$residuals, xlab="Fitted Values", ylab="Residuals")
```



'stab.glu', 'chol', 'age', 'ratio', and 'time.ppn' appears to be the most important predictors as suggested by the stepwise selection summary above. 'stab.glu', 'age', and 'time.ppn' are also indicated as the important predictors by the linear regression summary. 'stab.glu' appears to be the most important of all.

The residuals against fitted values plot shows that the error variances are fairly constant with a few potential outliers. One might argue that the variability increases with larger fitted values, but we will leave it as is for now and move on to the next.

d-)

```
f.rt <- rpart(glyhb ~ ., data=diabetes2)
f.rt
```

```
## n= 366
##
## node), split, n, deviance, yval
##       * denotes terminal node
##
##  1) root 366 1816.85800  5.607295
##    2) stab.glu< 158 324  546.30990  5.028333
##      4) stab.glu< 105.5 268  306.23520  4.806007
```

```
##        8) age< 55.5 205  115.01170  4.598683 *
##        9) age>=55.5 63  153.73920  5.480635 *
##      5) stab.glu>=105.5 56  163.43180  6.092321
##       10) time.ppn< 105 22   27.56829  5.169546 *
##       11) time.ppn>=105 34  105.00860  6.689412
##         22) bp.1s< 141.5 16   40.96785  5.881875 *
##         23) bp.1s>=141.5 18   44.33236  7.407222 *
##    3) stab.glu>=158 42  324.14380 10.073570
##      6) chol< 195.5 17  106.69140  8.470000 *
##      7) chol>=195.5 25  144.01200 11.164000
##       14) bp.1d< 86.5 11   49.21207  9.935455 *
##       15) bp.1d>=86.5 14   65.15250 12.129290 *
```

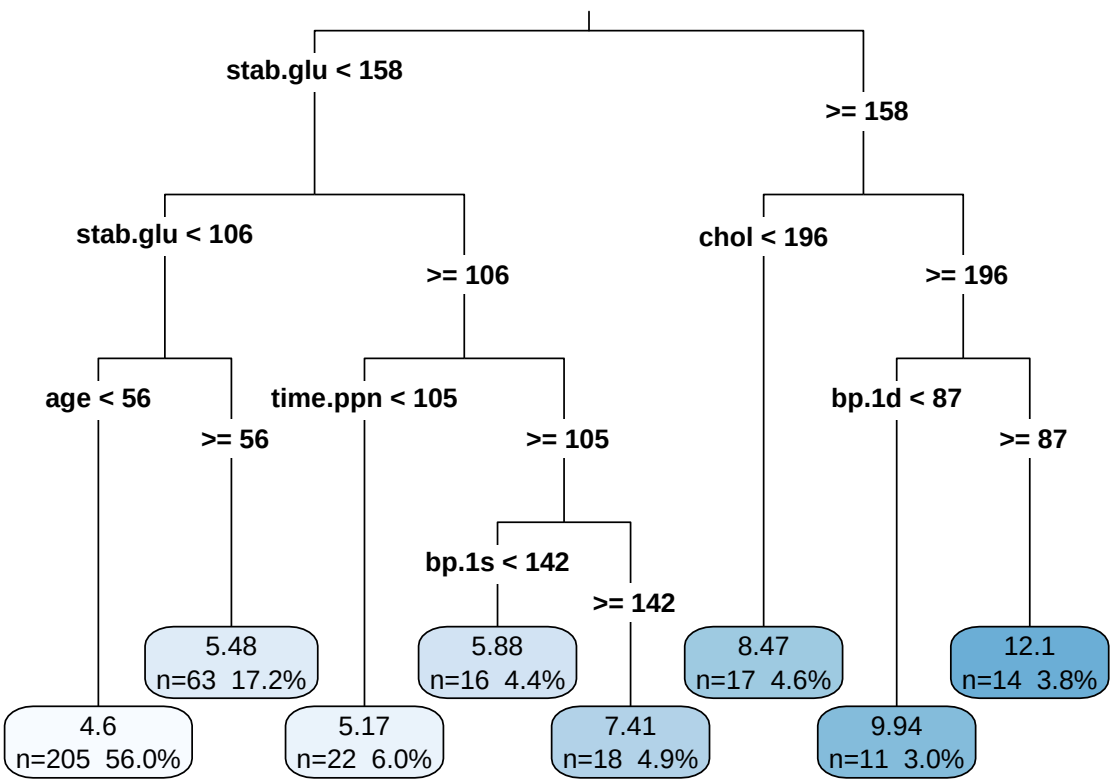
```r
median(diabetes2$glyhb)
```

```
## [1] 4.86
```

For 'stab.glu < 158', there are 324 observations, and the mean response for them is 5.028333. The terminal node is indicated with * on the right-hand side in the results shown above, and the node#8 'age < 55.5' is the largest with 205 observations and the mean response of 4.598683, which is close to the median of 'glyhb' (4.86).

e-)

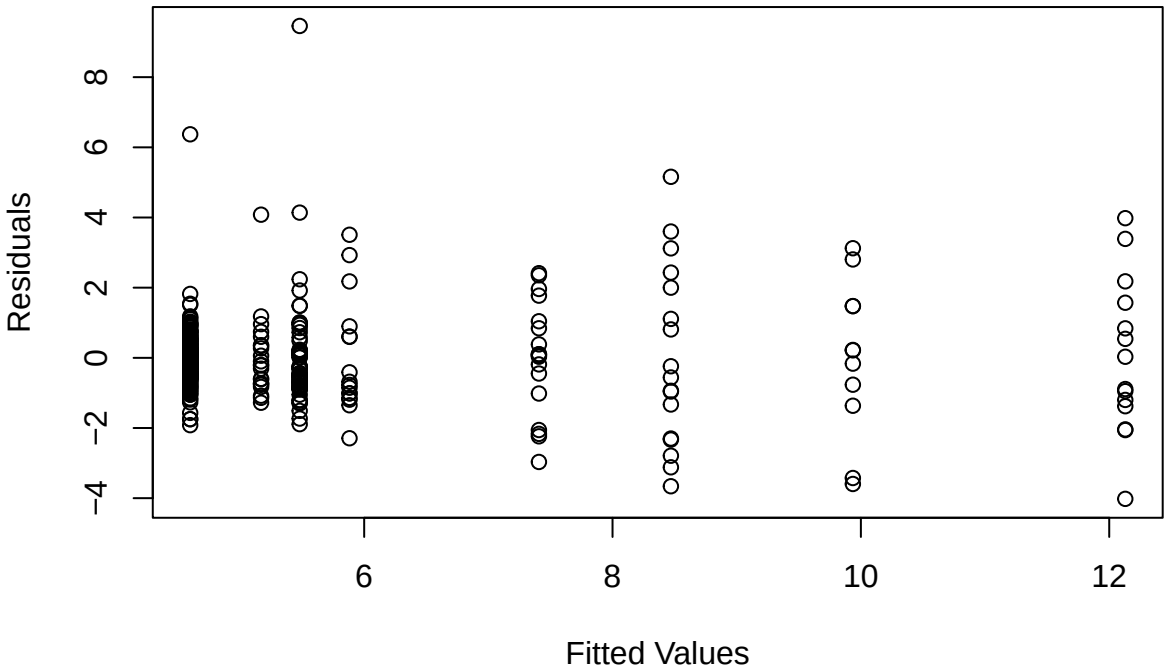
```r
rpart.plot(f.rt, digits = 3, fallen.leaves = TRUE, type = 3, extra = 101)
```



The predictor that splits the tree at the top is the most important predictor. In our case, 'stab.glu' is the most important, and it splits the data into two categories 'stab.glu < 158' and 'stab.glu >= 158'.

f-)

```r
yhat.rt <- predict(f.rt, diabetes2)
ei <- diabetes2$glyhb - yhat.rt
plot(yhat.rt, ei, xlab="Fitted Values", ylab="Residuals")
```




```r
MAE <- function(actual, predicted) {mean(abs(actual - predicted))}
mae1 <- MAE(diabetes2$glyhb, yhat.rt)
mae2 <- MAE(diabetes2$glyhb, f1$fitted.values)

SSE <- function(actual, predicted) {sum((actual - predicted)^2)}
sse1 <- SSE(diabetes2$glyhb, yhat.rt)
sse2 <- SSE(diabetes2$glyhb, f1$fitted.values)

R2 <- function(actual, predicted) {sum((actual - predicted)^2)/((length(actual)-1)*var(actual))}
rsq1 <- 1-R2(diabetes2$glyhb, yhat.rt)
rsq2 <- 1-R2(diabetes2$glyhb, f1$fitted.values)

tbl1 <- data.frame(rbind(c(mae1,mae2), c(sse1,sse2), c(rsq1,rsq2)))
colnames(tbl1) <- c("Regresstion Tree", "Linear Regression")
rownames(tbl1) <- c("MAE", "SSE", "Rsquared")
kable(tbl1, digits=3, align="c", caption="Regression Tree vs. Linear Regression",
      booktabs=TRUE) %>% kable_styling(latex_options="HOLD_position")
```

Table 1: Regression Tree vs. Linear Regression

	Regresstion Tree	Linear Regression
MAE	0.824	0.910
SSE	602.675	720.659
Rsquared	0.668	0.603

The residuals against the fitted values plot shows the constant error variances and looks a little bit better than

the one for the linear regression model. We would choose the regression tree model over the linear regression model because as shown in the table above, MAE and SSE is lower and R^2 is higher for the regression tree model.

Problem 2

a-)

```
mush <- getDataSet("Mushroom.csv")
mush$Class <- as.factor(mush$Class)
mush$veil.type <- NULL # removed because all the observations are the same.
mush$stalk.root <- NULL # removed because of too many observations marked as '?'
# Tried including 'stalk.root' and no difference in the result
set.seed(1023)
train.index <- createDataPartition(y = mush$Class, p = 0.9, list = FALSE)
train.set <- mush[train.index,]
test.set <- mush[-train.index,]
```

b-)

```
tree.model <- C5.0(train.set[,-1], train.set[,1])
tree.model
```

```
##
## Call:
## C5.0.default(x = train.set[, -1], y = train.set[, 1])
##
## Classification Tree
## Number of samples: 7313
## Number of predictors: 20
##
## Tree size: 9
##
## Non-standard options: attempt to group attributes
```

```
summary(tree.model)
```

```
##
## Call:
## C5.0.default(x = train.set[, -1], y = train.set[, 1])
##
##
## C5.0 [Release 2.07 GPL Edition]      Thu Nov 11 03:01:56 2021
## -------------------------------
##
## Class specified by attribute `outcome'
##
## Read 7313 cases (21 attributes) from undefined.data
##
## Decision tree:
##
## odor in {p,f,c,y,s,m}: p (3416)
## odor in {a,l,n}:
## :...spore.print.color = r: p (65)
##     spore.print.color in {k,n,u,h,w,o,b,y}:
##     :...stalk.color.below.ring in {b,c}: e (0)
##         stalk.color.below.ring = y: p (20)
```

```
##         stalk.color.below.ring = n:
##         :...stalk.surface.above.ring in {s,f,y}: e (56)
##         :   stalk.surface.above.ring = k: p (16)
##         stalk.color.below.ring in {w,p,g,e,o}:
##         :...gill.size = b: e (3512)
##             gill.size = n:
##             :...population in {s,n,a,v,y}: e (220)
##                 population = c: p (8)
##
##
## Evaluation on training data (7313 cases):
##
##      Decision Tree
##    ----------------
##    Size      Errors
##
##       8    0( 0.0%)   <<
##
##
##     (a)   (b)    <-classified as
##    ----  ----
##    3788          (a): class e
##          3525    (b): class p
##
##
##  Attribute usage:
##
##  100.00% odor
##   53.29% spore.print.color
##   52.40% stalk.color.below.ring
##   51.14% gill.size
##    3.12% population
##    0.98% stalk.surface.above.ring
##
##
## Time: 0.0 secs
```

The decision tree shown above clearly indicates which sub-tree leads to 'p'(poisonous) and 'e'(edible).

c-)

```
pred.train <- predict(tree.model, train.set)
CrossTable(train.set$Class, pred.train, prop.chisq = FALSE, prop.c = FALSE, prop.r = FALSE, dnn = c('ac
```

```
##
##
##    Cell Contents
## |-------------------------|
## |                       N |
## |         N / Table Total |
## |-------------------------|
##
##
## Total Observations in Table:  7313
##
##
```

```
##                | predicted default
## actual default |         e |         p | Row Total |
## ---------------|-----------|-----------|-----------|
##              e |      3788 |         0 |      3788 |
##                |     0.518 |     0.000 |           |
## ---------------|-----------|-----------|-----------|
##              p |         0 |      3525 |      3525 |
##                |     0.000 |     0.482 |           |
## ---------------|-----------|-----------|-----------|
##   Column Total |      3788 |      3525 |      7313 |
## ---------------|-----------|-----------|-----------|
##
##
```

The error rate is 0 since the table above shows that the model correctly predicts for all the observations of both 'e' and 'p'.

d-)

```
pred.test <- predict(tree.model, test.set)
CrossTable(test.set$Class, pred.test, prop.chisq = FALSE, prop.c = FALSE, prop.r = FALSE, dnn = c('actua
```

```
##
##
##    Cell Contents
## |-------------------------|
## |                       N |
## |         N / Table Total |
## |-------------------------|
##
##
## Total Observations in Table:  811
##
##
##                | predicted default
## actual default |         e |         p | Row Total |
## ---------------|-----------|-----------|-----------|
##              e |       420 |         0 |       420 |
##                |     0.518 |     0.000 |           |
## ---------------|-----------|-----------|-----------|
##              p |         0 |       391 |       391 |
##                |     0.000 |     0.482 |           |
## ---------------|-----------|-----------|-----------|
##   Column Total |       420 |       391 |       811 |
## ---------------|-----------|-----------|-----------|
##
##
```

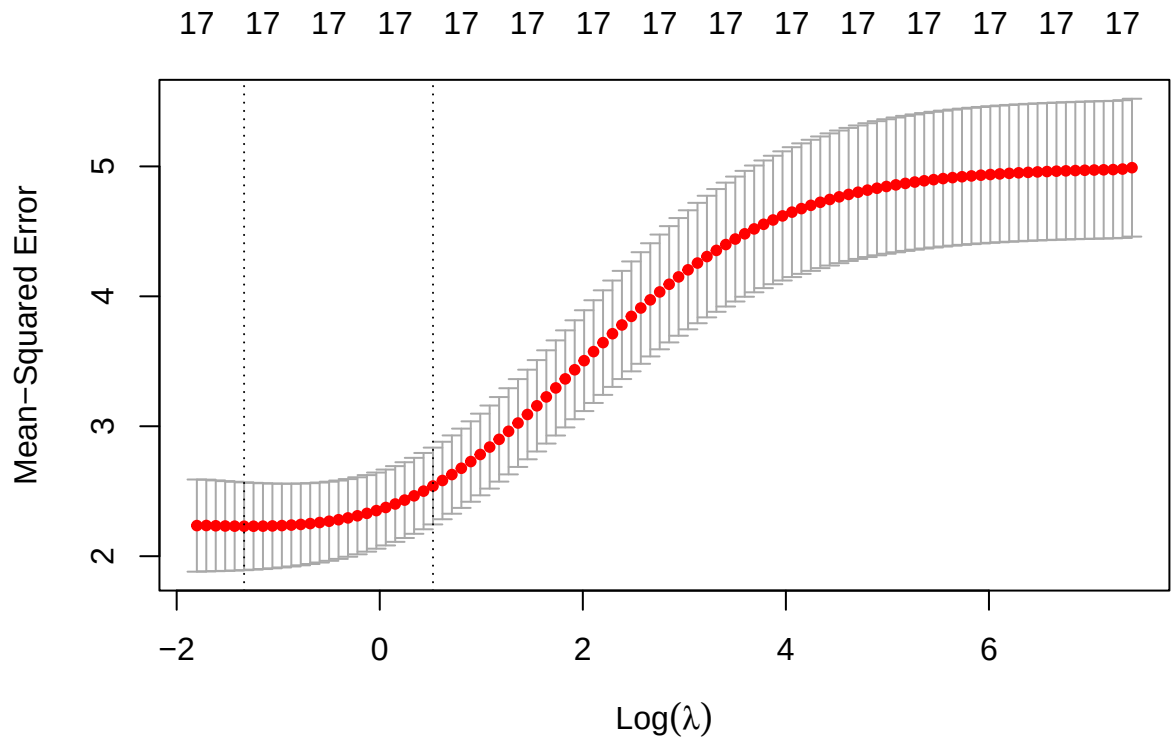
The model also correctly predicts all the observations in the test data, so the model is very stable.

Problem 3

a-)

```
x <- model.matrix(glyhb~., diabetes2)[,-c(1)]
y <- diabetes2$glyhb
RidgeMod <- glmnet(x, y, alpha=0, nlambda=100,lambda.min.ratio=0.0001)
```

```r
CvRidgeMod <- cv.glmnet(x, y, alpha=0, nlambda=100,lambda.min.ratio=0.0001)
plot(CvRidgeMod)
```

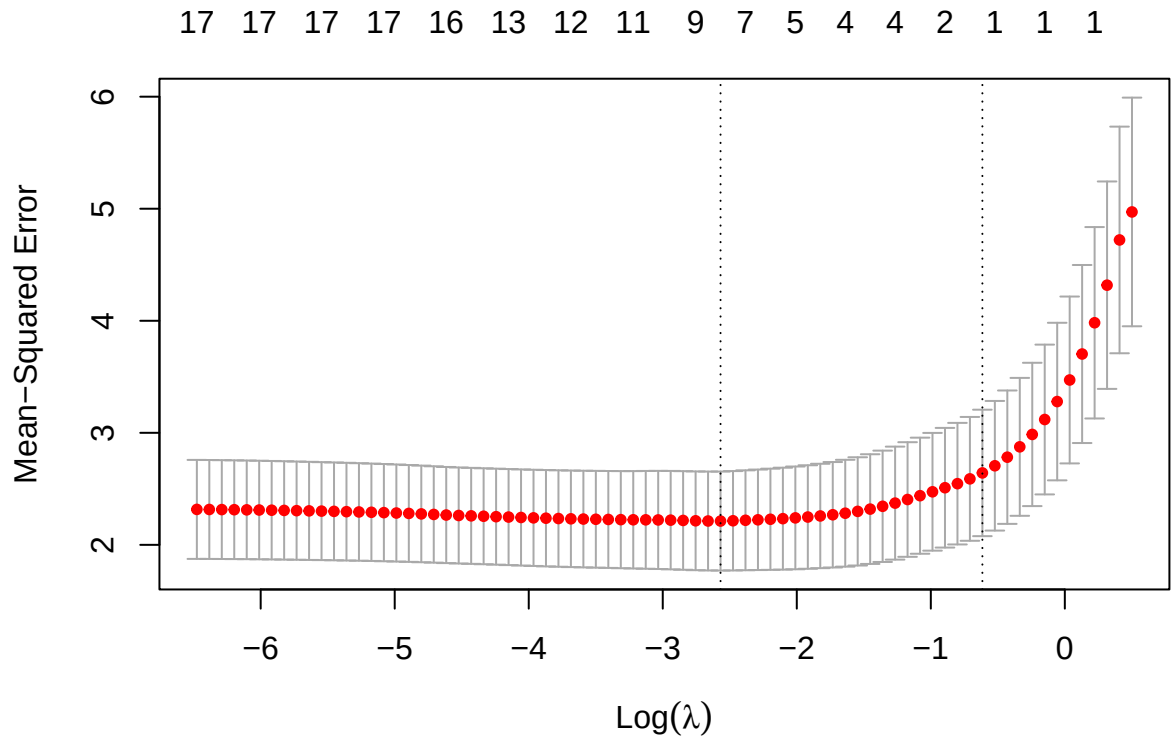



```r
best.lambda.ridge <- CvRidgeMod$lambda.min
pred.ridge <- coef(CvRidgeMod, s = "lambda.min")
pred.ridge
```

```
## 18 x 1 sparse Matrix of class "dgCMatrix"
##                              s1
## (Intercept)    -1.328227777553
## id              0.000005396313
## chol            0.004451908935
## stab.glu        0.024331250362
## hdl            -0.002404828450
## ratio           0.107348044381
## locationLouisa -0.250637388191
## age             0.014657849342
## genderfemale    0.091032101832
## height          0.016011948060
## weight         -0.001269075296
## framemedium     0.077378231722
## framelarge     -0.143550951348
## bp.1s           0.003288735201
## bp.1d          -0.002034801677
## waist           0.021117179887
## hip             0.005975694213
## time.ppn        0.000496399942
```

b-)

```r
LassoMod <- glmnet(x, y, alpha=1, nlambda=100,lambda.min.ratio=0.0001)
CvLassoMod <- cv.glmnet(x, y, alpha=1, nlambda=100,lambda.min.ratio=0.0001)
plot(CvLassoMod)
```




```r
best.lambda.lasso <- CvLassoMod$lambda.min
pred.lasso <- coef(CvLassoMod, s = "lambda.min")
pred.lasso
```

```
## 18 x 1 sparse Matrix of class "dgCMatrix"
##                            s1
## (Intercept)     0.7038941082
## id              .
## chol            0.0036515235
## stab.glu        0.0266669093
## hdl             .
## ratio           0.0944867288
## locationLouisa -0.0817644964
## age             0.0115524523
## genderfemale    .
## height          .
## weight          .
## framemedium     0.0090054324
## framelarge      .
## bp.1s           0.0003117938
## bp.1d           .
## waist           0.0056787862
## hip             .
## time.ppn        0.0002765628
```

c-)

```r
EnetMod <- glmnet(x, y, alpha=0.5, nlambda=100,lambda.min.ratio=0.0001)
CvElasticnetMod <- cv.glmnet(x, y,alpha=0.5,nlambda=100,lambda.min.ratio=0.0001)
best.lambda.enet <- CvElasticnetMod$lambda.min
pred.enet <- coef(CvElasticnetMod, s = "lambda.min")
pred.enet
```

```
## 18 x 1 sparse Matrix of class "dgCMatrix"
##                            s1
## (Intercept)     0.5584168528
## id              .
## chol            0.0037035876
## stab.glu        0.0259800144
## hdl             .
## ratio           0.1006663601
## locationLouisa -0.1038324280
## age             0.0121280945
## genderfemale    .
## height          .
## weight          .
## framemedium     0.0303930692
## framelarge      .
## bp.1s           0.0007488016
## bp.1d           .
## waist           0.0079199724
## hip             .
## time.ppn        0.0003062365
```

d-)

```r
rr.huber <- rlm(glyhb ~ ., data = diabetes2, psi = psi.huber)
summary(rr.huber)
```

```
##
## Call: rlm(formula = glyhb ~ ., data = diabetes2, psi = psi.huber)
## Residuals:
##     Min      1Q  Median      3Q     Max
## -8.1649 -0.5762 -0.0310  0.5509 10.1066
##
## Coefficients:
##                Value   Std. Error t value
## (Intercept)    -1.3656  1.5923    -0.8576
## id              0.0000  0.0000    -0.0304
## chol            0.0034  0.0022     1.5167
## stab.glu        0.0295  0.0010    29.0507
## hdl            -0.0016  0.0069    -0.2238
## ratio           0.0627  0.0761     0.8245
## locationLouisa -0.2456  0.1147    -2.1408
## age             0.0092  0.0040     2.2841
## genderfemale    0.0810  0.1686     0.4803
## height          0.0170  0.0204     0.8330
## weight         -0.0022  0.0035    -0.6244
## framemedium     0.0131  0.1337     0.0978
## framelarge     -0.0983  0.1781    -0.5518
## bp.1s           0.0047  0.0032     1.4533
## bp.1d          -0.0043  0.0050    -0.8477
```

```
## waist           0.0240  0.0205     1.1727
## hip             0.0086  0.0234     0.3692
## time.ppn        0.0004  0.0002     2.5695
##
## Residual standard error: 0.8283 on 348 degrees of freedom
```

```
rr.bisquare <- rlm(glyhb ~ ., data = diabetes2, psi = psi.bisquare)
```

```
## Warning in rlm.default(x, y, weights, method = method, wt.method = wt.method, :
## 'rlm' failed to converge in 20 steps
```

```
summary(rr.bisquare)
```

```
##
## Call: rlm(formula = glyhb ~ ., data = diabetes2, psi = psi.bisquare)
## Residuals:
##      Min       1Q   Median       3Q      Max
## -7.52746 -0.47725  0.03264  0.60474 10.18557
##
## Coefficients:
##                Value   Std. Error t value
## (Intercept)    -0.1506  1.5397    -0.0978
## id              0.0000  0.0000    -0.2816
## chol            0.0044  0.0022     2.0340
## stab.glu        0.0273  0.0010    27.7174
## hdl            -0.0070  0.0067    -1.0410
## ratio          -0.0366  0.0736    -0.4982
## locationLouisa -0.2469  0.1109    -2.2257
## age             0.0074  0.0039     1.8985
## genderfemale    0.0450  0.1631     0.2758
## height          0.0106  0.0197     0.5375
## weight         -0.0018  0.0034    -0.5343
## framemedium    -0.0061  0.1293    -0.0470
## framelarge      0.0201  0.1722     0.1166
## bp.1s           0.0045  0.0031     1.4202
## bp.1d          -0.0036  0.0049    -0.7484
## waist           0.0248  0.0198     1.2517
## hip             0.0060  0.0226     0.2656
## time.ppn        0.0003  0.0002     1.9701
##
## Residual standard error: 0.8079 on 348 degrees of freedom
```

e-)

```
yhat.ridge <- predict(RidgeMod, s = best.lambda.ridge, newx = x)
yhat.lasso <- predict(LassoMod, s = best.lambda.lasso, newx = x)
yhat.enet <- predict(EnetMod, s = best.lambda.enet, newx = x)
yhat.huber <- predict(rr.huber, diabetes2)
yhat.bisquare <- predict(rr.bisquare, diabetes2)

mae.ridge <- MAE(diabetes2$glyhb, yhat.ridge)
mae.lasso <- MAE(diabetes2$glyhb, yhat.lasso)
mae.enet <- MAE(diabetes2$glyhb, yhat.enet)
mae.huber <- MAE(diabetes2$glyhb, yhat.huber)
mae.bisquare <- MAE(diabetes2$glyhb, yhat.bisquare)
```

```r
sse.ridge <- SSE(diabetes2$glyhb, yhat.ridge)
sse.lasso <- SSE(diabetes2$glyhb, yhat.lasso)
sse.enet <- SSE(diabetes2$glyhb, yhat.enet)
sse.huber <- SSE(diabetes2$glyhb, yhat.huber)
sse.bisquare <- SSE(diabetes2$glyhb, yhat.bisquare)

rsq.ridge <- 1-R2(diabetes2$glyhb, yhat.ridge)
rsq.lasso <- 1-R2(diabetes2$glyhb, yhat.lasso)
rsq.enet <- 1-R2(diabetes2$glyhb, yhat.enet)
rsq.huber <- 1-R2(diabetes2$glyhb, yhat.huber)
rsq.bisquare <- 1-R2(diabetes2$glyhb, yhat.bisquare)

ridge.pf <- c(mae.ridge, sse.ridge, rsq.ridge)
lasso.pf <- c(mae.lasso, sse.lasso, rsq.lasso)
enet.pf <- c(mae.enet, sse.enet, rsq.enet)
huber.pf <- c(mae.huber, sse.huber, rsq.huber)
bisquare.pf <- c(mae.bisquare, sse.bisquare, rsq.bisquare)

tbl2 <- cbind(tbl1, ridge.pf, lasso.pf, enet.pf, huber.pf, bisquare.pf)
colnames(tbl2) <- c("Regression Tree","OLS","Ridge","Lasso","ElasticNet","Huber","Bisquare")
kable(tbl2, digits=3, align="c", caption="Comparison of Model Performance",
      booktabs=TRUE) %>% kable_styling(latex_options="HOLD_position")
```

Table 2: Comparison of Model Performance

	Regression Tree	OLS	Ridge	Lasso	ElasticNet	Huber	Bisquare
MAE	0.824	0.910	0.925	0.918	0.919	0.887	0.889
SSE	602.675	720.659	731.129	739.765	738.220	737.117	766.538
Rsquared	0.668	0.603	0.598	0.593	0.594	0.594	0.578

The regression tree performs the best, so we would choose it as the final model.